

# SUBJECT- DWPD(3350702)

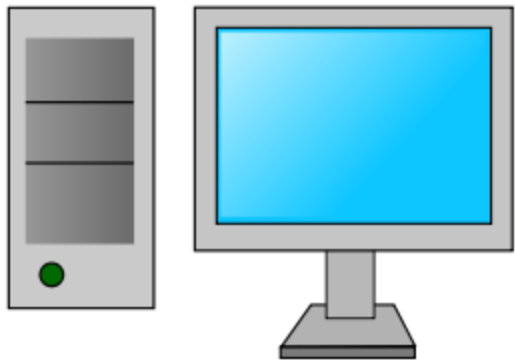
## COOKIES AND SESSION (UNIT-4)



PREPARED BY- N. K. SHUKLA(LECT., COMPUTER DEPARTMENT, AVPTI)

Local Computer

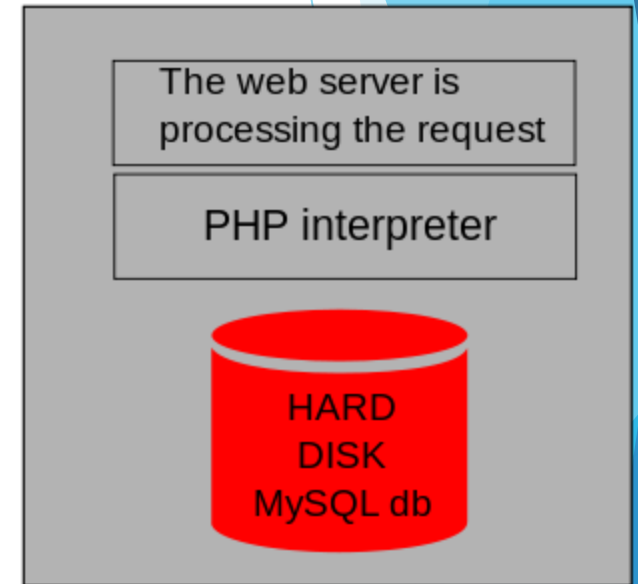
<http://www.example.com/login.php>



Internet



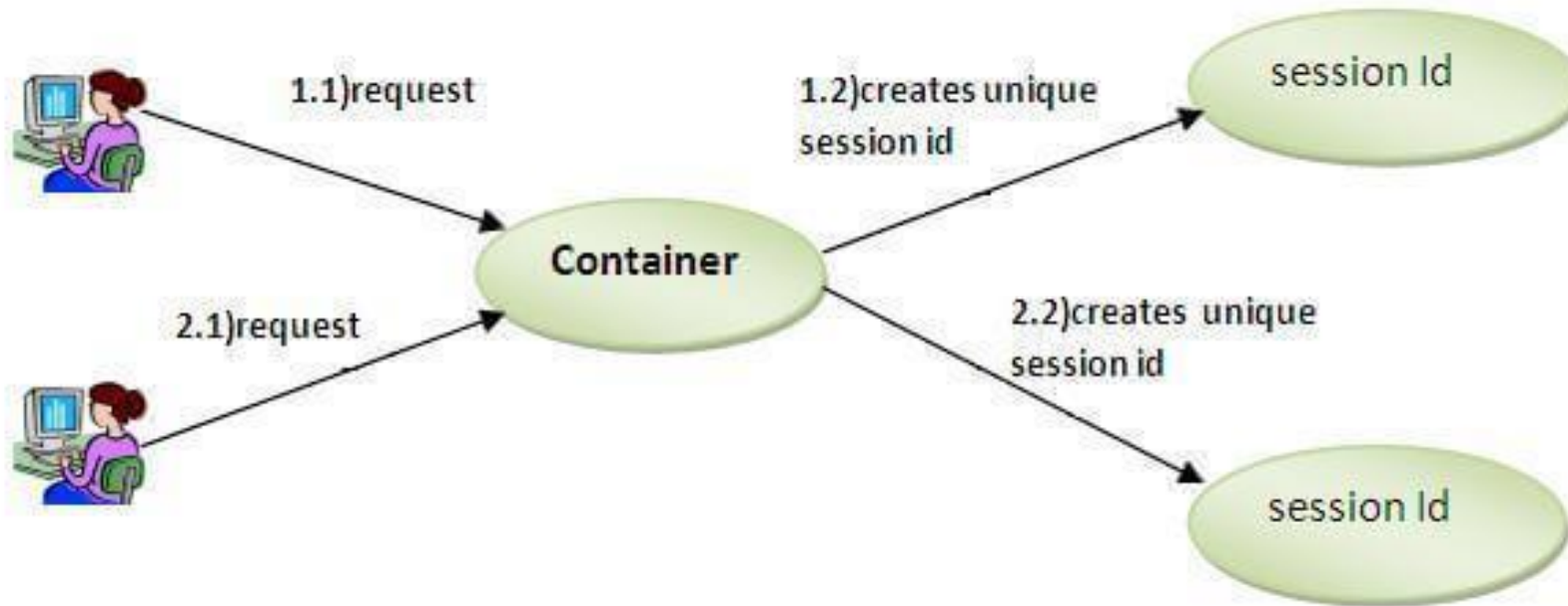
Web server



page in html

# Session

- ▶ **Definition:** “ Session is a global variable store in a server, it creates a file in a temporary directory on the server where registered session variables and their values are stored. ”



# Session

- ▶ The location of the temporary file is determined by a setting in the **php.ini** file called **session.save\_path**. Before using any session variable make sure you have setup this path.
- ▶ When a session is started following things happen –
  - 1) PHP first creates a unique identifier for that particular session which is a random string of 32 hexadecimal numbers such as 3c7foj34c3jj973hjkop2fc937e3443.
  - 2) A cookie called **PHPSESSID** is automatically sent to the user's computer to store unique session identification string.
  - 3) A file is automatically created on the server in the designated temporary directory and bears the name of the unique identifier prefixed by sess\_iesess\_3c7foj34c3jj973hjkop2fc937e3443.
- ▶ When a PHP script wants to retrieve the value from a session variable, PHP automatically gets the unique session identifier string from the PHPSESSID cookie and then looks in its temporary directory for the file bearing that name and a validation can be done by comparing both values.
- ▶ A session ends when the user loses the browser or after leaving the site, the server will terminate the session after a predetermined period of time, commonly 30 minutes duration.

# Start a Session

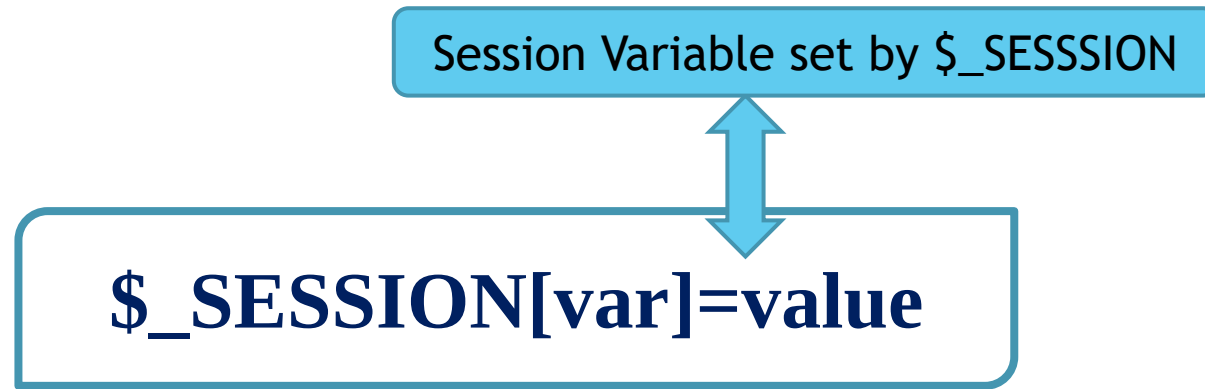
- ▶ A session is started with the **session\_start()** function.
- ▶ Session variables are set with the PHP global variable: `$_SESSION`.
- ▶ Syntax-

**session\_start();**

- ▶ The `session_start()` function must be the very first thing in your document. Before any HTML tags.
- ▶ A PHP session is easily started by making a call to the `session_start()` function. This function first checks if a session is already started and if none is started then it starts one. It is recommended to put the call to `session_start()` at the beginning of the page.
- ▶ Session variables are stored in associative array called `$_SESSION[]`. These variables can be accessed during lifetime of a session.
- ▶ Make use of **isset()** function to check if session variable is already set or not.

# Creating a Session

- ▶ In order to create a session, you must first call the PHP `session_start()` function and then store your values in the `$_SESSION` array variable.
- ▶ Syntax:



- ▶ For Example:-

```
$_SESSION["uid"]="avpt";  
$_SESSION["pass"] = "avpt602";
```

## EXAMPLE: Get user's name and password and store in session

#starting and creating a session

```
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>
<h1> Login Form</h1>

<form method="POST">
    <h2> Enter User Name:-
    <input type="text" name="unm"><br>
    <h2> Enter User Name:-
    <input type="text" name="unm"><br>
    <input type="submit" name="login"
value="Login">
</form>
```

```
<?php
```

```
If(isset($_POST["login"]))
{
```

```
// Set session variables
```

```
$_SESSION["uname"] = $_POST["unm"];
$_SESSION["pass"] = $_POST["pass"];
echo "Session variables are set.";
?>
```

```
</body>
```

```
</html>
```

# Accessing a session:-

**EXAMPLE:** Get user's name and password and store in session

## #Accessing Session variable on another page..

```
<?php
// Start the session
session_start();

If (isset($_SESSION["uname"]) and isset($_SESSION["uname"]))
{

    echo "<h1> Welcome ".$_SESSION["uname"];
}

?>
```



# Destroy and Deleting a Session:-

- ▶ A PHP session can be destroyed by **session\_destroy()** function. This function does not need any argument and a single call can destroy all the session variables. If you want to destroy a single session variable then you can use **unset()** function to unset a session variable.

- ▶ Here is the example to unset a single variable –

```
<?php
    unset($_SESSION['counter']);
?>
```

- ▶ Here is the call which will destroy all the session variables –

```
<?php
    session_destroy();
?>
```

## ▶ Turning on Auto Session

You don't need to call `start_session()` function to start a session when a user visits your site if you can set `session.auto_start` variable to 1 in `php.ini` file.

## ▶ Sessions without cookies

- ▶ There may be a case when a user does not allow to store cookies on their machine. So there is another method to send session ID to the browser.
- ▶ Alternatively, you can use the constant `SID` which is defined if the session started. If the client did not send an appropriate session cookie, it has the form `session_name=session_id`. Otherwise, it expands to an empty string. Thus, you can embed it unconditionally into URLs.

- ▶ The following example demonstrates how to register a variable, and how to link correctly to another page using SID.

```
<?php
    session_start();

    if (isset($_SESSION['counter'])) {
        $_SESSION['counter'] = 1;
    }else {
        $_SESSION['counter']++;
    }

    $msg = "You have visited this page ". $_SESSION['counter'];
    $msg .= "in this session.";

    echo ( $msg );
?>
<p>
    To continue click following link <br />

    <a href = "nextpage.php?<?php echo htmlspecialchars(SID); ?>">
</p>
```

# Difference between Cookies and Session

## Comparison

### Cookies

Cookies are **stored in the user's browser**

A cookie can keep information in the user's browser until deleted by user or set as per the timer. It will not be **destroyed even if you close the browser**.

**Cookies can only store string**

We can **save cookie** for future reference

### Session

Sessions are stored in **server**

A session is available as long as the browser is opened. User can't disable the session. It will be **destroyed if you close the browser**

Can store not only strings but also **objects**

**session can't be.**

The background features abstract, overlapping geometric shapes in various shades of blue, ranging from light sky blue to deep navy blue. These shapes are primarily located on the right side of the image, creating a modern, layered effect. The rest of the background is a solid, very light blue.

**Thank You**